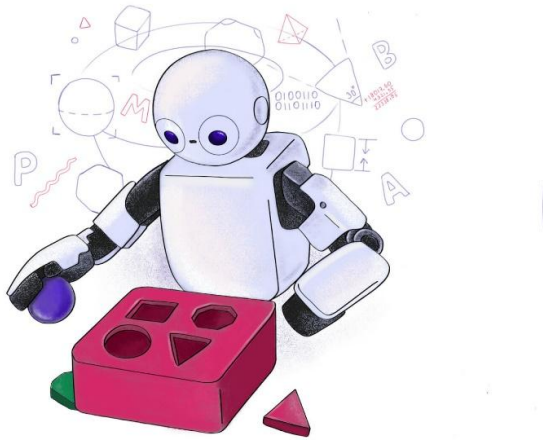
TP558 - Tópicos avançados em Machine Learning

# *Human-level control through deep reinforcement learning*

Controle em nível humano por aprendizagem por reforço profundo



Inatel

João Roberto Teixeira dos Santos
joao.Santos@mtel.inate.br

# Abertura

## Como ensinar um agente a agir bem vendo apenas pixels?

### Contexto
Antes do DQN, muitos agentes de RL dependiam de atributos projetados à mão ou de estados de baixa dimensão.

### Problema
Q-learning com uma rede neural não linear pode oscilar ou divergir: os dados são correlacionados e o próprio alvo muda durante o treinamento.

### Motivação
Aprender uma política end-to-end a partir de imagens, usando uma única receita de arquitetura e hiperparâmetros em tarefas muito diferentes.

**Atari 2600 como laboratório: 49 jogos**
- **Entrada visual**
- **Ações discretas**
- **Recompensas ao longo do tempo**

# Introdução do artigo

## Objetivo

Criar um agente capaz de aprender políticas de controle diretamente de entradas sensoriais de alta dimensão, sem engenharia manual de atributos.

## Hipótese

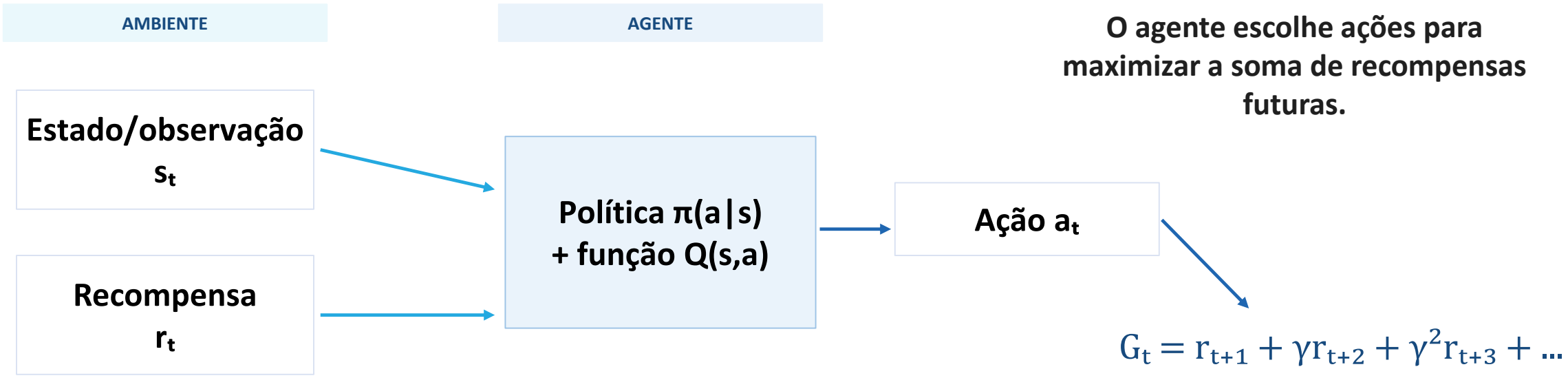
Uma rede convolucional profunda pode aproximar Q(s,a), desde que o treinamento seja estabilizado com experience replay e uma target network separada.

## Contribuições

- Deep Q-Network (DQN) end-to-end
- Mesma arquitetura/hiperparâmetros em 49 jogos
- Comparação com humanos e métodos anteriores
- Análise dos componentes centrais

**Ponto-chave: “uma única arquitetura” não significa um único modelo multitarefa — uma rede separada foi treinada para cada jogo.**

# Fundamentação teórica: aprendizado por reforço




**O agente escolhe ações para maximizar a soma de recompensas futuras.**

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

γ = 0,99 no artigo

No Atari:

- s = Pilha de 4 frames
- a = Comando do joystick
- r = Variação da pontuação

# Fundamentação teórica: Q-learning → DQN

**$Q^*(s,a)$ = retorno futuro máximo esperado após tomar a ação a no estado s**

$$Q^*(s,a) = \max\pi \; E[\, \Sigma \, \gamma^k \, r_{t+k+1} \mid s_t=s, \, a_t=a \,]$$

**Ideia do Q-learning**

Aprender Q(s,a) e, em cada estado, preferir a ação com maior valor. O DQN usa uma CNN como aproximador de Q.

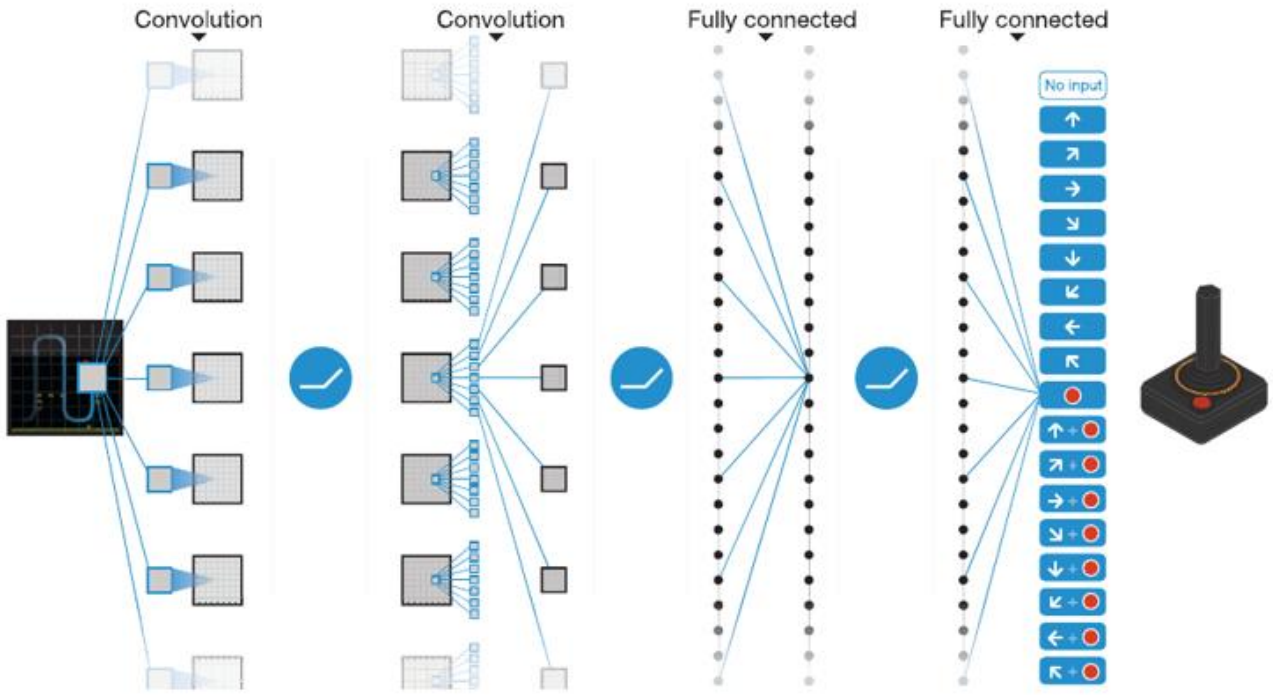
**Por que isso fica instável com redes profundas?**

- Observações consecutivas são fortemente correlacionadas.
- Atualizar Q muda a política e, portanto, a distribuição dos dados.
- O alvo usa valores Q que também estão sendo atualizados.

**Resposta do DQN**

**experience replay + target network congelada**

# Arquitetura da Deep Q-Network (DQN)




*Fonte: Mnih et al. (2015), Figura 1.*

**Entrada**
84 × 84 × 4 (quatro frames pré-processados)

**Conv 1**
32 filtros 8×8 • stride 4 • ReLU

**Conv 2 + Conv 3**
64 filtros 4×4 s2 → 64 filtros 3×3 s1

**Fully connected**
512 unidades ReLU

**Saída**
Um Q-value por ação válida (4 a 18 ações)

**Uma única passada calcula Q para todas as ações.**

# Metodologia: cenário experimental

**49**
jogos Atari 2600

**50 M**
frames de treino por jogo

**30**
episódios de avaliação

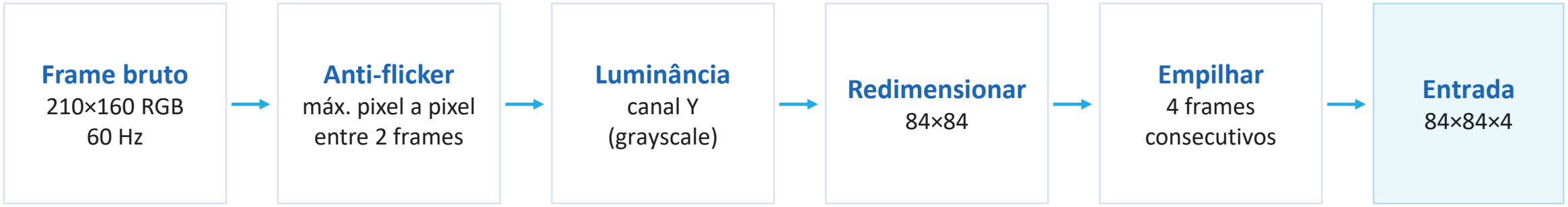
**≤ 5 min**
por episódio avaliado

### Receita fixa
Mesma arquitetura, algoritmo e hiperparâmetros em todos os jogos. Porém, uma rede diferente é treinada para cada jogo.

### Conhecimento prévio mínimo
O agente sabe que recebe imagens, conhece o número de ações e usa a contagem de vidas para delimitar episódios. Ele não recebe semântica dos comandos.

**Vídeo original: 210 × 160 pixels, 128 cores, 60 Hz**

# Metodologia: pré-processamento da observação




## Por que empilhar frames?

Um único frame não informa direção ou velocidade. A sequência de quatro frames fornece contexto temporal curto.

## Frame skipping

A ação escolhida é repetida por 4 frames. Isso reduz custo de decisão e acelera a interação com o emulador.

**Resultado: menor dimensionalidade + informação temporal suficiente para o controle.**

# Metodologia: os dois mecanismos de estabilização

## 1 Experience replay

**Replay memory D**

- Guarda as experiências recentes (capacidade: 1.000.000 frames)
- Amostra minibatches aleatórios de 32 transições
- Quebra correlações temporais
- Reutiliza experiências e suaviza mudanças na distribuição

## 2 Target network

**Q com parâmetros congelados**

- Uma rede separada calcula os alvos de Q-learning
- Pesos são copiados da rede online a cada C = 10.000 updates
- Entre cópias, o alvo fica fixo
- Reduz o feedback imediato que causa oscilações/divergência

**Intuição: aprender com amostras menos correlacionadas e perseguir um alvo que não se move a cada passo.**

# Metodologia: fluxo do algoritmo DQN

**1** **Escolher ação**
ε-greedy: aleatória com prob. ε; caso contrário argmax Q(s,a)

**2** **Interagir**
Executar a ação e observar recompensa r e próximo frame

**3** **Armazenar**
Salvar (s, a, r, s') na replay memory

**4** **Amostrar**
Sortear minibatch aleatório de transições

**5** **Calcular alvo**
Usar a target network para obter y

**6** **Atualizar**
Minimizar erro entre y e Qonline; sincronizar target periodicamente

## Alvo de Q-learning

$$y = r \quad \text{se terminal}$$
$$y = r + \gamma \max_{a'} Qtarget(s',a') \quad \text{caso contrário}$$

## Função de perda

$$L = ( y - Qonline(s,a) )^2$$

Detalhes adicionais do artigo:
- reward clipping para −1, 0 e +1
- clipping do erro TD em [−1, +1]
- atualização via RMSProp

# Treinamento e avaliação: parâmetros que importam

## Treinamento

| | |
|---|---|
| **Replay memory** | 1.000.000 frames |
| **Minibatch** | 32 |
| **γ** | 0,99 |
| **Learning rate** | 0,00025 (RMSProp) |
| **Exploração ε** | 1,0 → 0,1 em 1 M frames |
| **Replay start** | 50.000 frames |

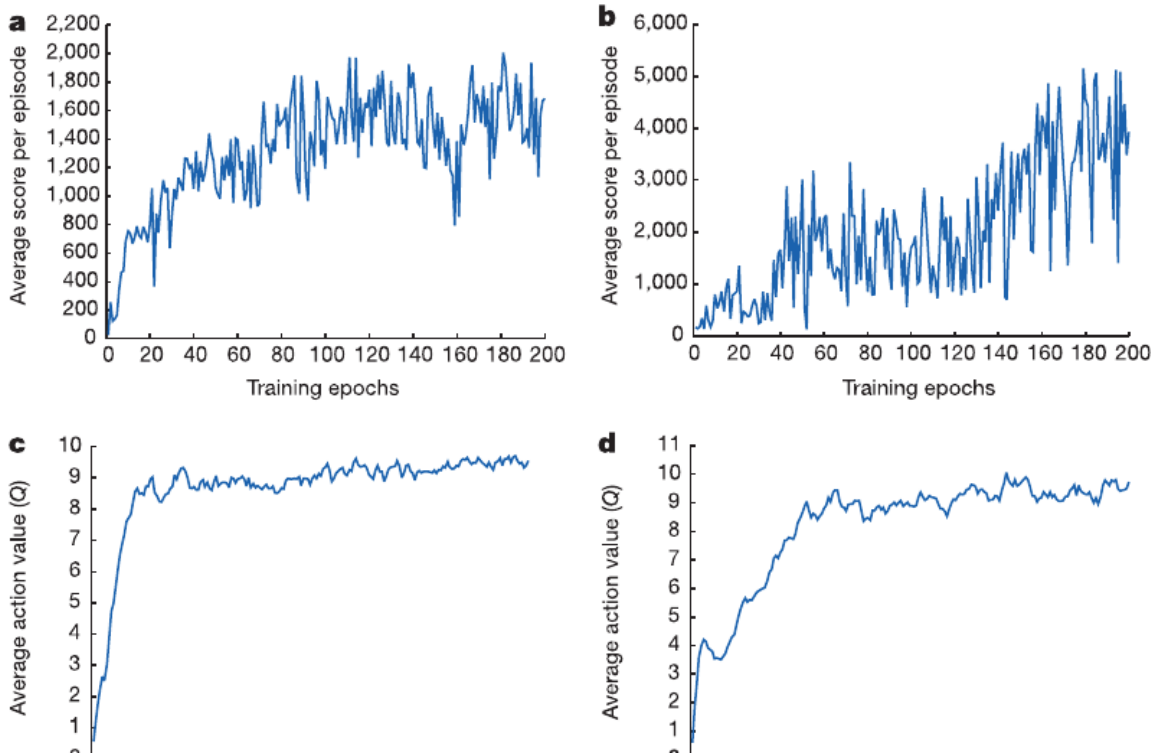
## Avaliação

| | |
|---|---|
| **Política** | ε-greedy, ε = 0,05 |
| **Episódios** | 30 por jogo |
| **Duração** | máx. 5 min |
| **Condições iniciais** | 0 a 30 no-op actions |
| **Humano** | 20 episódios após 2 h de prática/jogo |
| **Áudio** | desabilitado |

**Treino principal: 50 milhões de frames por jogo ≈ 38 dias de experiência de jogo.**

A recompensa é recortada durante o treinamento para {−1, 0, +1}; isso facilita usar a mesma taxa de aprendizado, mas perde informação de magnitude.

# Resultados: o treinamento realmente estabiliza?




*Fonte: Mnih et al. (2015), Figura 2.*

## Score médio

Tende a crescer ao longo do treino, mas permanece ruidoso — comportamento esperado em RL.

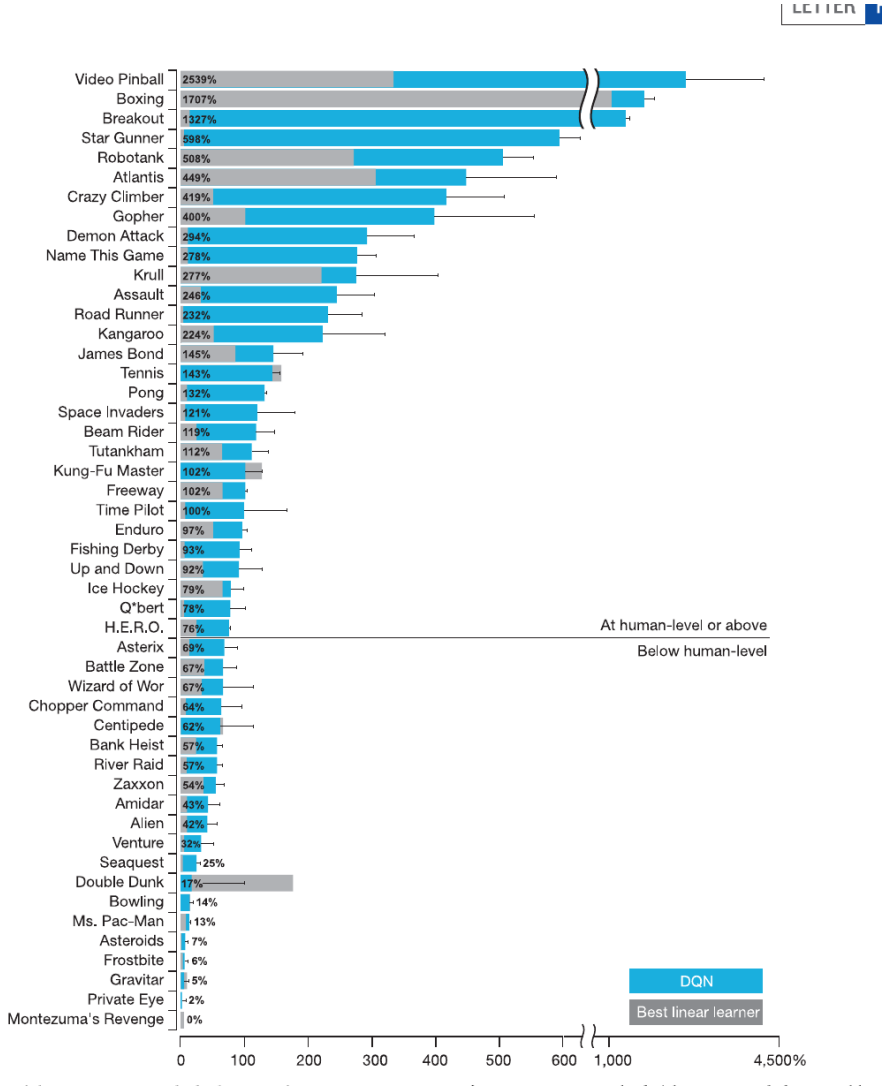
## Q médio previsto

Cresce no início e depois fica relativamente estável nos estados de validação.

## Leitura

Evidência de que redes grandes puderam ser treinadas com sinal de reforço sem divergência catastrófica.

# Resultados: desempenho nos 49 jogos




*Fonte: Mnih et al. (2015), Figura 3.*

**43/49**
melhor que os métodos de RL anteriores

**29/49**
≥ 75% do humano

## Normalização

$$100 \times (\text{DQN} - \text{Random}) / (\text{Humano} - \text{Random})$$

0% = agente aleatório
100% = jogador humano profissional
“human-level” no artigo: ≥ 75%

**A grande contribuição não é vencer todos os jogos: é usar a mesma receita para tarefas muito diferentes.**

# Resultados: onde o DQN funciona — e onde falha

## Muito acima da referência humana

**2539%**
Video Pinball

**1708%**
Boxing

**1327%**
Breakout

## Muito abaixo da referência humana

**0%**
Montezuma's Revenge

**2,5%**
Private Eye

**6,2%**
Frostbite

### O que funcionou

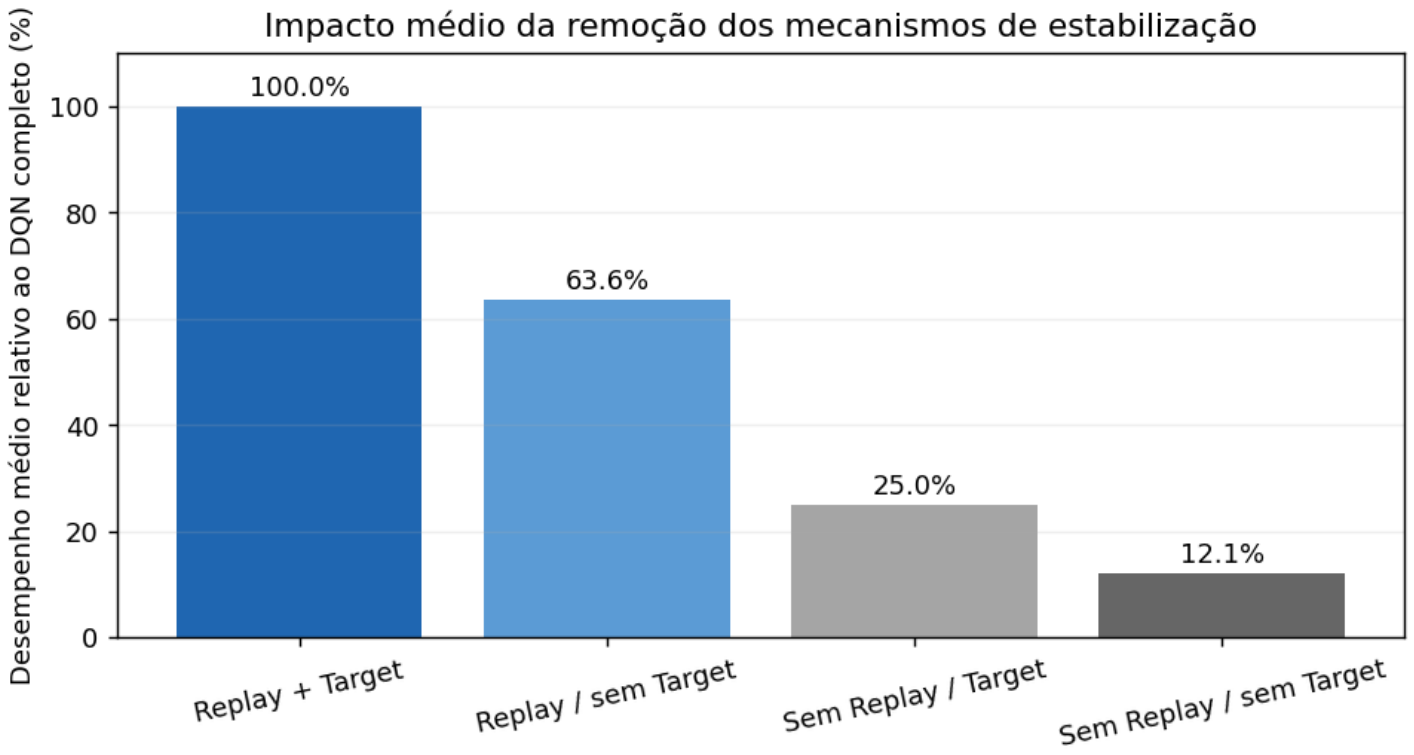
A arquitetura aprende estratégias eficazes em jogos muito diferentes. Em Breakout, por exemplo, aprende a abrir um túnel lateral para colocar a bola atrás dos blocos.

### O que não funcionou

O próprio artigo destaca que tarefas com planejamento temporal mais longo continuam difíceis. Montezuma's Revenge é o caso extremo: o agente obtém 0% normalizado.

**Leitura crítica: desempenho "humano" agregado não significa competência uniforme.**

# Resultados: replay e target network são realmente necessários?




**100%**
Replay + Target

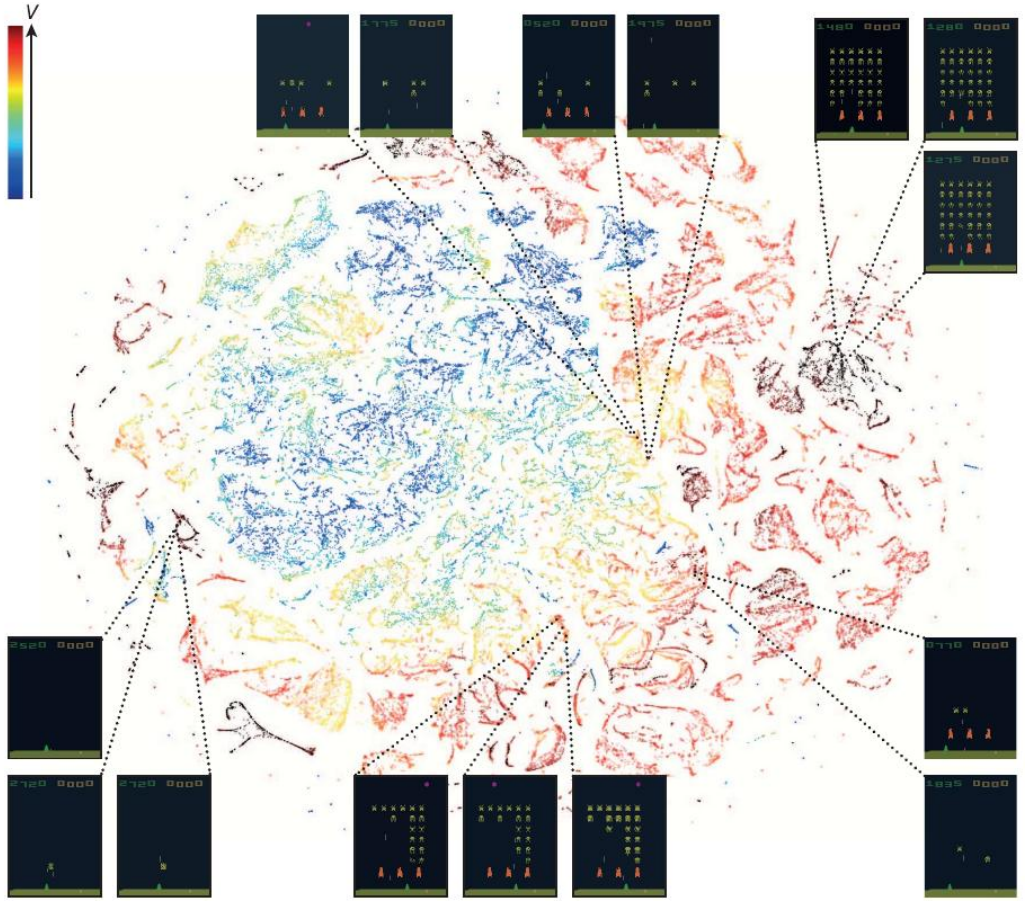
**63,6%**
Replay / sem Target

**25,0%**
Sem Replay / Target

**Sem Replay e sem Target: 12,1%**

**Em todos os 5 jogos da análise, a configuração completa obtém o melhor score.**

# Resultados: o que a rede aprende internamente?




*Fonte: Mnih et al. (2015), Figura 4 (t-SNE, Space Invaders).*

## Visualização

t-SNE das ativações da última camada escondida para estados visitados pelo agente.

## Padrão observado

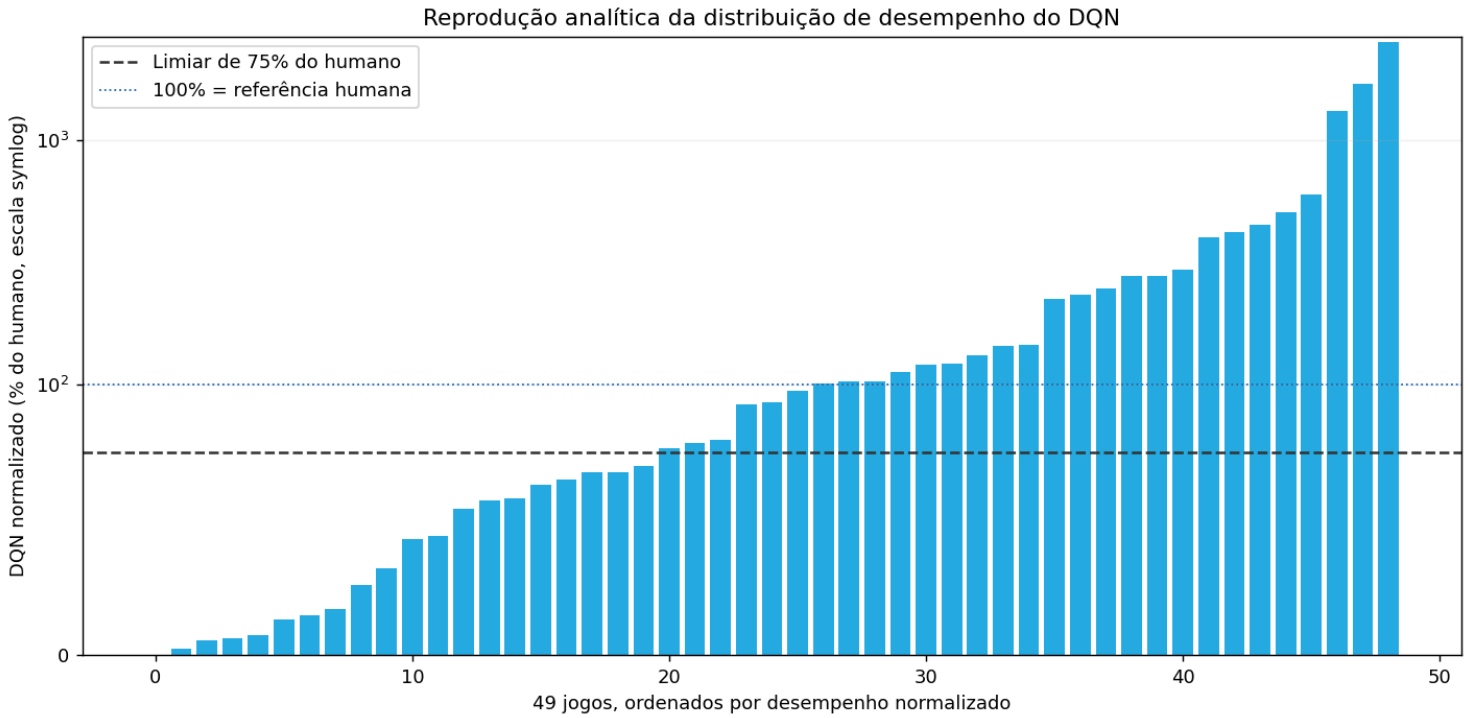
Estados perceptualmente parecidos tendem a ficar próximos; estados diferentes também podem se aproximar quando têm valor previsto semelhante.

## Interpretação

A representação deixa de ser apenas visual e passa a refletir características úteis para decisão e previsão de recompensa.

*Importante: t-SNE é uma visualização; não prova "compreensão" causal do jogo.*

# Exemplo prático: reprodução parcial dos resultados




## O que o notebook faz

- Recalcula a normalização em exemplos
- Verifica 29/49 jogos ≥ 75% do humano
- Reproduz a análise replay/target

## O que o notebook NÃO faz

Não re-treina os agentes Atari por 50 milhões de frames. Isso exigiria muito mais tempo e infraestrutura.

*Notebook incluído no pacote; dados da Extended Data Table 2.*

# Slide extra — leitura do código

## 1. Normalização

```
examples = pd.DataFrame({
    "Game": ["Space Invaders", "Breakout", "Montezuma's Revenge"],
    "Random": [148.0, 1.7, 0.0],
    "Human": [1652.0, 31.8, 4367.0],
    "DQN": [1976.0, 401.2, 0.0],
})
examples["Normalized_calculated_pct"] = 100 * (examples["DQN"] - examples["Random"]) / (examples["Human"] - examples["Random"])
examples.round(1)
```

## 2. Validação 29/49

```
human_level = (df["Normalized_DQN_pct"] >= 75).sum()
print(f"Jogos com DQN >= 75% do desempenho humano: {human_level}/{len(df)}")
assert human_level == 29
```

## 3. Estudo de ablação

```
ablation = pd.DataFrame({
    "Game": ["Breakout", "Enduro", "River Raid", "Seaquest", "Space Invaders"],
    "Replay + Target": [316.8, 1006.3, 7446.6, 2894.4, 1088.9],
    "Replay sem Target": [240.7, 831.4, 4102.8, 822.6, 826.3],
    "Sem Replay + Target": [10.2, 141.9, 2867.7, 1003.0, 373.2],
    "Sem Replay e sem Target": [3.2, 29.1, 1453.0, 275.8, 302.0],
})
ablation
```

Mostrar saída oculta

```
configs = ["Replay + Target", "Replay sem Target", "Sem Replay + Target", "Sem Replay e sem Target"]
rel = ablation.copy()
baseline = ablation["Replay + Target"].copy()
for c in configs:
    rel[c] = 100 * ablation[c] / baseline
summary = rel[configs].mean().round(1)
summary
```

### 1. Normalização

Converte cada jogo para uma escala comparável: 0% = agente aleatório e 100% = referência humana.

### 2. Validação 29/49

Conta quantos jogos atingem pelo menos 75% do desempenho humano. O assert confirma exatamente 29 dos 49 jogos.

### 3. Estudo de ablação

Usa Replay + Target como 100% e mede quanto do desempenho permanece quando cada mecanismo é removido.

**Mensagem principal: o notebook reproduz analiticamente os resultados publicados — ele não re-treina o DQN.**

# Análise crítica do trabalho

## Pontos fortes

- Aprendizado end-to-end diretamente dos pixels
- Mesma receita em 49 tarefas diferentes
- Estudo de análises dos mecanismos centrais
- Comparação com humanos e métodos anteriores
- Evidência de representações internas úteis

## Limitações

- 50 M frames por jogo: baixa eficiência amostral
- Uma rede separada por jogo
- Reward clipping perde magnitude
- Replay uniforme trata transições como igualmente importantes
- Dificuldade em planejamento temporal estendido

## Possíveis melhorias

Replay priorizado — sugerido pelos próprios autores —, exploração melhor em recompensas raras, maior eficiência amostral e transferência entre tarefas;
Planejamento a longo prazo;
Maior eficiência e tranferência.

## Aplicações práticas

Controle a partir de sensores de alta dimensão: robótica, simuladores, jogos, controle visual e problemas sequenciais de decisão.

# Conclusão

### 1. Representação
A CNN aprende características diretamente dos pixels e as orienta para prever valores de ação — sem atributos específicos de cada jogo.

### 2. Estabilidade
Experience replay e target network tornam viável treinar Q-learning com redes profundas em um domínio visual complexo.

### 3. Desempenho
O DQN atinge desempenho próximo ou superior ao humano em muitos jogos, mas continua fraco em tarefas de planejamento longo.

**Impacto: o trabalho consolidou uma receita prática para Deep Reinforcement Learning end-to-end.**

# Perguntas?

# Referências


1. MNIH, V. et al. Human-level control through deep reinforcement learning. Nature, v. 518, p. 529–533, 2015. DOI: 10.1038/nature14236.

2. SUTTON, R. S.; BARTO, A. G. Reinforcement Learning: An Introduction. MIT Press, 1998.

3. WATKINS, C. J. C. H.; DAYAN, P. Q-learning. Machine Learning, v. 8, p. 279–292, 1992.

4. BELLEMARE, M. G. et al. The Arcade Learning Environment: An Evaluation Platform for General Agents. JAIR, v. 47, p. 253–279, 2013.

5. LIN, L.-J. Reinforcement Learning for Robots Using Neural Networks. Technical Report, 1993.

6. VAN DER MAATEN, L.; HINTON, G. Visualizing Data using t-SNE. JMLR, v. 9, p. 2579–2605, 2008.


Todas as referências acima são citadas pelo artigo ou utilizadas diretamente para explicar os conceitos do seminário.

# Obrigado!

**Quiz do seminário:**

**https://forms.cloud.microsoft/Pages/ResponsePage.aspx?id=USyefs-fLk-nmwl-UqQkzUyY0qIxJGlMnBH3YP-fAcxUNkJUNlQ4UU03MkE1Tzg3TDNQTldMRUZTUi4u**




Inatel